

Lab-7: Finite difference in Python for 2D problems

The aim of this lab is to apply the Finite Difference Method to find approximate solutions to Partial Differential Equations (PDE) in 2D.

Instructions:

- Clone your repository from the UPPA gitlab: it contains only the class `Mesh2D` for the mesh.
- Do one .py file per exercise and add it to the deposit.
- The figures posted in your report must be added to your git repository and the report must point to the image produced by your code (see the example).
- It is **not required** to do an exhaustive answer question by question. The questions only serve to guide you on the work requested.

Exercises:

Exercise 1. Mesh class

Create a Mesh class similar to Mesh already used previously in 1D.

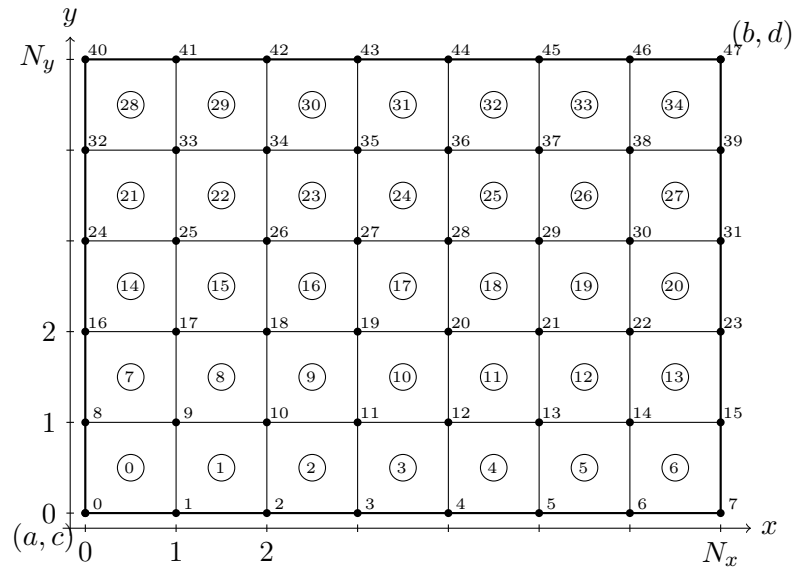


Figure 5.1: Cartesian 2D mesh of a rectangular 2D domain $[a; b] \times [c; d]$, with points numbers and cells numbers (circles).

This class must define:

- the total number of points `NPoints`;
- the total number cells `NCells`;

- the points coordinates **Points** must be defined in a list of 2-values line by line;
- the cells **Cells** are to be defined as a list of 4-coordinates defining the connectivity of the cell (each integer coordinate of the constitutive points) in counter-clockwise from lowest point (bottom left);
- mapping $I(i, j)$ must be defined to compute the index in points order from an integer coordinate;
- **Init(f)** initialization function for evaluating the 2-variables function f given on each point of the mesh, returning the 1D vector values.

Note that the points ordering (line by line) is an arbitrary choice that enforce the order of vector u_h . The connectivity (counter-clockwise from bottom left point) of the cells is also a arbitrary choice. An appropriate usage of Mesh class for 1D discretization in both direction is recommended.

Unit test for this class is given.

Example python script is given to illustrate the expected usage of Mesh class. The **abcd** attribute and **to2Dimg** member are given below:

```
class Mesh:
    def __init__(self, a, b, c, d, Nx, Ny):
        """2D discrete mesh for [a;b]x[c;d] domain using (Nx+1)*(Ny+1) points."""
        self.abcd = (a,b,c,d)
        ....

    ....

    def to2Dimg(self, uh):
        """Converts a discretization on the mesh to be plotted as image."""
        return uh.reshape((self.Ny+1, self.Nx+1))[:-1,:]
```

Exercise 2. 2D heat equation

Consider the following 2D heat equation on rectangular domain $\Omega =]a; b[\times]c; d[$, and diffusion coefficient $\kappa > 0$.

$$\begin{cases} \frac{\partial u}{\partial t} = \kappa \Delta u & \text{on } \Omega \times]0; T] \\ u(0, x, y) = u_0(x, y) & \text{on } \Omega \\ BC & \text{on }]0; T] \end{cases}$$

Paper work

1. Write a 2nd order in space and 2nd order in time Crank-Nicholson consistent with this equation. This scheme must be unconditionnaly stable (proof not asked).
2. Choose a setup (non trivial initial condition u_0 and boundary conditions) together with an exact solution to later test the program and perform a convergence study. Justify.

Programming work

1. Plot your initial condition.
2. Implement the scheme and plot your solution at $t = T$.

3. Draw a convergence study in space and time.